LAB_09

→ Objectives

The objective of this lab is to introduce you to signal handling in UNIX!

→ Description

Signals are software interrupts that provide a mechanism to deal with asynchronous events (e.g., a user pressing Control-C to interrupt a program that the shell is currently executing). A signal is a notification to a process that an event has occurred that requires the attention of the process (this typically interrupts the normal flow of execution of the interrupted process). Signals can also be used as a synchronization technique or even as a simple form of interprocess communication. Signals could be generated by hardware interrupts, the program itself, other programs, the OS kernel, or by the user.

All these signals have a unique symbolic name and starts with SIG. The standard signals (also called POSIX reliable signals) are defined in the header file <*signal.h*>. Here are some examples:

- SIGINT: Interrupt a process from keyboard (e.g., pressing Control-C). The process is terminated.
- SIGQUIT: Interrupt a process to quit from keyboard (e.g., pressing Control-/). The process is terminated and a core file is generated.
- SIGTSTP: Interrupt a process to stop from keyboard (e.g., pressing Control-Z). The process is stopped from executing.
- SIGUSR1/SIGUSR2: These are user-defined signals, for use in application programs.

After a signal is generated, it is delivered to a process to perform some action in response to this signal. Since signals are asynchronous events, a process must decide ahead of time how to respond when a particular signal is delivered. There are three different options possible when a signal is delivered to a process:

1) Perform the default action. Most signals have a default action associated with them, if the process does not change the default behavior, then the default action specific to that particular signal will occur. The predefined default signal is specified as SIG_DFL.
2) Ignore the signal. If a signal is ignored, then the default action is performed. The predefined ignore signal handler is specified as SIG_IGN.
3) Catch and Handle the signal. It is also possible to override the default action and invoke a specific user-defined task when a signal is received. This is usually performed by invoking a signal handler using *signal()* or *sigaction()* system calls.

However, the signals SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.

Now, let's write a program to handle a simple signal that catches either of the two user-defined signals and prints the signal number.

☒ Create a user-defined function (signal handler) …

- Invoked when a signal is received

☒ Call the above user-defined signal.

For calling the signal, we use an infinite loop with the *pause* function. The pause function causes the calling process to sleep until a signal is received. We use the printf statements in the signal handler from the user-defined function, however, printf is not a reentrant function and is not recommended to be used in signal handlers, you should use write instead.

☒ Compile the file and run the program using the following example:

$ gcc -Wall -o <executable_name> <file_name>

$ ./<executable_name> &

→

$

$ jobs

→

$ kill -USR1 %1

→

$

$ kill -USR2 %1

→

$

$ kill -SIGUSR1 <pid>

→

$ kill -STOP <pid>

→

$

$ jobs

→

$ kill -USR1 %1

→

$ kill -USR2 %1

→

$ kill -CONT <pid>

→

→

$

$ jobs

→

$ kill -TERM %1

$

→

$ jobs

$

(→) output: observe the output as you provide these command-line arguments to stdin …

In the example above we used the *kill* command to generate the signal. The kill command can also be used to terminate a process using the TERM signal (this is the default signal if no signal is specified), the kill command could be used to generate any other signal that is supported by the kernel. You can list all signals that can be generated using *kill -l*.

→ Homework

Modify the program forkexecvp.c …

- ☒ when you type Ctrl+C or Ctrl+Z the child process is interrupted or suspended.
- ☒ the parent process should continue to wait until it received a quit signal (Ctrl+\)